

2025/2026

[illegible]

1

- As 2 classes `MultipleRewardGame` e `SingleScoreGame` (bem como `AbstractGame`, a superclasse abstrata destas duas últimas), todas implementam o interface `Game`;
Isto permite ter métodos que têm um parâmetro do tipo `Game` e podem receber um objeto de qualquer dos sub-tipos concretos de `Game`, como é o caso do método `playGame` da classe `PCOMain`.
- Os enumerados `Hearts`, `Balls`, e `Signs` implementam o interface `Symbol`;
A classe `PlayArea` tem vários atributos do tipo `Symbol`: a matriz `grid`, que é uma matriz de `Symbol`'s, o símbolo `empty`, que é o símbolo adotado para representar as posições vazias, e um *array* `symbols` guardando a coleção dos símbolos que se podem usar na `grid`; a classe `Game` também tem atributos deste tipo;
Isto permite que se possam variar os símbolos usados em cada jogo – basta atribuir ao *array* de `Symbols`, usado na criação de um jogo, o conjunto de valores do tipo de símbolos desejado (ver instruções 19, 50 e 80 da classe `PCOMain`).
- A classe `PlayArea` tem um atributo do tipo `Eliminator` e outro do tipo `Accomodator` os quais irão referenciar, em tempo de execução, instâncias de uma das possíveis implementações destes interfaces:
 - `SimpleEliminator` e `PatternEliminator`, implementações do interface `Eliminator` e
 - `SimpleAccomodator` e `FatalFallAccomodator`, implementações do interface `Accomodator`.
 Isto permite que se usem objetos de qualquer daquelas classes em todas as situações em que se pretende usar um eliminador ou um acomodador (ver na classe `PCOMain`).

O que se pretende de vós neste projeto?

Neste projeto a vossa tarefa é construir, em Java, vários dos tipos de dados apresentados no diagrama de classes fornecido, e que são necessárias para executar o programa da classe `PCOMain`, dada por nós.

No método `main` desta classe:

- É criado um `SimpleEliminator`, que é um `Eliminator`; é também criado um `SimpleAccomodator`, que é um `Accomodator`;
- É criado um `MultipleRewardGame` com 2 linhas da *grid* já preenchidas (dificuldade 2) e que vai usar o `SimpleEliminator` anterior como eliminador, e o `SimpleAccomodator` anterior como acomodador;
- O utilizador joga o jogo;
- É criado um `SingleScoreGame` com 4 linhas da *grid* já preenchidas (dificuldade 4) e que vai usar o `SimpleEliminator` e o `SimpleAccomodator` anteriores como eliminador e acomodador;
- O utilizador joga o jogo;
- É criado um `SingleScoreGame` com 0 linhas da *grid* já preenchidas (dificuldade 0); pelo facto de a *grid* começar completamente vazia, este jogo é considerado terminado logo desde o início;

- Finalmente é criada uma lista com três padrões e um `PatternEliminator`; é criado também um `FatalFallAccomodator`;
- É criado um `MultipleRewardGame` com 1 linha da grid já preenchida (dificuldade 1) e que vai usar o `PatternEliminator` anterior como eliminador, e o `FatalFallAccomodator` anterior como acomodador;
- O utilizador joga o jogo.

Os seguintes enumerados, interfaces e classes já são dados por nós (não é para fazer!):

- **Symbol**: *interface* representando os símbolos que podem ser usados em jogos;
- **Hearts**, **Balls** e **Signs** enumerados que implementam o *interface* `Symbol`;
- **Piece**: classe cujas instâncias representam peças de jogo;
- **PlayArea**: classe que representa áreas contendo símbolos onde se vão colocando novas peças e se vão eliminando e acomodando símbolos. É nesta classe que são chamados os métodos do eliminador e do acomodador;
 - O método `eliminateAccomodateAll` na classe `PlayArea` permite, numa mesma jogada, fazer eliminações alternadas em linhas e em colunas, seguidas das acomodações necessárias, até não haver mais nada para eliminar nessa jogada; este método usa os métodos privados `eliminate` e `accomodate`;
 - O método privado `eliminate` da `PlayArea` invoca o método `eliminateSequence` do *Eliminator* – será passada por parâmetro uma linha da grid quando for eliminação horizontal, e uma coluna da grid quando for para eliminação vertical;
 - O método privado `accomodate` da `PlayArea` invoca o método `accomodate` do *Accomodator* sobre cada uma das colunas da *grid* (que são o mesmo que as linhas da matriz transposta da *grid*);
- **FatalFallAccomodator**: classe que implementa o *interface* `Accomodator`; nesta classe, o método `accomodate` acomoda uma sequência de símbolos de modo a “fechar” os “buracos” existentes nessa sequência, sendo que, se são acomodados mais do que um determinado número de símbolos, os símbolos acima desses também desaparecem (“morrem por queda fatal”);
- **PatternEliminator**: classe que implementa o *interface* `Eliminator`; nesta classe, o método `eliminateSequence` elimina qualquer ocorrência de determinadas sequências de símbolos (chamadas padrões) definidas no seu construtor.

Estas duas últimas classes servem somente para que os alunos vejam que podem existir outras opções de escolha de eliminadores e acomodadores.

Para que o método `main` da classe `PCOMain` funcione como descrito acima, os alunos terão que construir os seguintes tipos de dados.

Os interfaces:

- **Game**, que define a constante:

- o `static final int SIZE_OF_PIECE = 3` que define o número de símbolos das peças a jogar;

e os métodos:

- o `int linesInGrid()` e `int colsInGrid()` que devolvem, respetivamente, o número de linhas e colunas da área do jogo;
- o `void permutatePiece(int n)` que aplica uma permutação de `n` posições na peça corrente do jogo;
- o `void placePiece(int col)` que coloca na coluna `col` da área de jogo a peça corrente do jogo e aplica as necessárias eliminações e acomodações e calcula e atualiza a pontuação do jogo;
- o `void generatePiece()` que gera uma nova peça; esta nova peça passa a ser a peça corrente do jogo;
- o `int spaceInColumn(int col)` que devolve o número de espaços vazios na coluna `col` do jogo;
- o `boolean finished()` que devolve `true` se o jogo já terminou;
- o `String currentPiece()` que devolve uma representação textual da peça corrente do jogo;
- o `void registerPlayScore(List<Integer> eliminated)` que atualiza a pontuação do jogo com os resultados de uma jogada; a lista `eliminated` contém o número de símbolos que foram eliminados na área de jogo em várias fases de eliminações durante a jogada;
- o `int score()` que devolve a pontuação corrente do jogo;
- **Eliminator**, que define o método:
 - o `int eliminateSequence(Symbol[] seq, int blockSize, Symbol nothing)`, que procura uma sequência de símbolos contíguos no *array* `seq` e os substitui pelo símbolo `nothing`; devolve o número de símbolos eliminados.
- **Accomodator**, que define o método:
 - o `void accomodate(Symbol[] seq, Symbol nothing)`, que “acomoda” uma sequência de símbolos de modo a “fechar os buracos” que ela tem, ou seja, todas as sequências contendo o símbolo `nothing`.

A classe abstrata:

- **AbstractGame**, que define o que é comum a vários tipos de jogos. A classe implementa o *interface* **Game**;

Oferece o seguinte construtor público:

- o `AbstractGame(int r, int c, int diff, Symbol empty, Symbol[] values, Random gen, Eliminator elim, Accomodator acc)` que inicializa um novo objeto; Este objeto vai ter uma área de jogo com `r` linhas e `c` colunas, das quais `diff` linhas já vão estar preenchidas com símbolos (escolhidos a partir da coleção `values`) aleatoriamente gerados pelo gerador `gen`; este jogo deverá usar o eliminador `elim` e o acomodador `acc` em todas as jogadas;

os seguintes métodos concretos:

- o `int linesInGrid(), int colsInGrid(), void permutatePiece(int n), void placePiece(int col), void generatePiece(), int spaceInColumn(int col), String currentPiece();`
- o `boolean finished()` que devolve *true* se não existe nenhuma coluna onde ainda caiba uma peça;
- o `String toString()` que devolve a representação textual deste jogo;

deixando por implementar os restantes métodos definidos no *interface* Game (por isso é que esta classe tem que ser abstrata).

As classes concretas:

- **MultipleRewardGame**: subclasse de **AbstractGame**, cujas instâncias representam jogos cuja pontuação é calculada da forma explicada na Nota1 no fim deste enunciado, e que terminam quando não existe nenhuma coluna onde ainda caiba uma peça.

Oferece o seguinte construtor público:

- o `MultipleRewardGame(int r, int c, int diff, Symbol empty, Symbol[] values, Random gen, Eliminator elim, Accomodator acc)` que inicializa um novo objeto;

e os métodos concretos:

- o `void registerPlayScore(List<Integer> eliminated)` que atualiza a pontuação do jogo com os resultados de uma jogada; a lista `eliminated` contém o número de símbolos que foram eliminados na área de jogo em várias fases de eliminações durante a jogada; a pontuação é calculada da forma explicada na **Nota1** à frente neste enunciado;
- o `int score()` que devolve a pontuação corrente do jogo;

e redefine:

- o `String toString()` que devolve a representação textual deste jogo, contendo já as pontuações (ver outputs fornecidos).

- **SingleScoreGame**: subclasse de **AbstractGame**, cujas instâncias representam jogos cuja pontuação só contabiliza a realização das jogadas (não conta com eliminações feitas) e que terminam quando a área de jogo está completamente vazia.

Oferece o seguinte construtor público:

- o `SingleScoreGame(int r, int c, int diff, Symbol empty, Symbol[] values, Random gen, Eliminator elim, Accomodator acc)` que inicializa um novo objeto;

e os métodos concretos:

- o `void registerPlayScore(List<Integer> eliminated)` que atualiza a pontuação do jogo com os resultados de uma jogada; a lista `eliminated` contém o número de símbolos que foram eliminados na área de jogo em várias fases de

eliminações durante a jogada; a pontuação é calculada contabilizando somente a realização das jogadas (não conta com eliminações feitas – ver **Nota1**);

- o `int score()` que devolve a pontuação corrente do jogo;

e redefine:

- o `boolean finished()` que devolve *true* se a área de jogo está completamente vazia ou se não existe nenhuma coluna onde ainda caiba uma peça;
- o `String toString()` que devolve a representação textual deste jogo, contendo já a pontuação (ver outputs fornecidos).

- **SimpleEliminator**, que implementa o *interface* **Eliminator**. As instâncias desta classe sabem eliminar sequências de símbolos repetidos.

Oferece o seguinte construtor público:

- o `SimpleEliminator()` que inicializa um novo eliminador que vai saber eliminar sequências de símbolos repetidos, através do método `eliminateSequence`;

e implementa o método:

- o `int eliminateSequence(Symbol[] seq, int blockSize, Symbol nothing)`, que procura `blockSize` ou mais elementos iguais na sequência `seq` e substitui esses símbolos pelo símbolo `nothing`; devolve o número de símbolos “eliminados” (ver **Nota2**).

- **SimpleAccomodator**, que implementa o *interface* **Accomodator**. As instâncias desta classe sabem acomodar uma dada sequência de símbolos, de modo a ficarem sem “buracos”.

Oferece o seguinte construtor público:

- o `SimpleAccomodator()` que inicializa um novo acomodador que vai saber acomodar sequências de símbolos;

e implementa o método:

- o `void accomodate(Symbol[] seq, Symbol nothing)`, que acomoda a sequência `seq` de modo a “fechar” os “buracos”, ou seja, os elementos contendo `nothing` (ver **Nota2**);

NOTA 1: Estas são as regras a aplicar para o cálculo da pontuação no tipo de jogo `SimpleScoreGame`:

- Por cada jogada válida efetuada, o utilizador recebe 10 pontos.

No tipo de jogo `MultipleRewardGame`, acrescem estas outras regras:

- O jogador recebe, por símbolos eliminados, 200 pontos pelos 3 símbolos base acrescidos de 50 pontos por cada símbolo adicional (e.g. uma sequência de 4

símbolos ofereceria 250 pontos, uma sequência de 5 símbolos ofereceria 300 pontos).

- A eliminação de múltiplas sequências na mesma jogada oferece pontuação extra ao jogador. Caso o utilizador consiga eliminar mais que uma sequência numa só jogada, a segunda sequência eliminada valerá o dobro dos pontos, a terceira o triplo e assim por diante. Por exemplo, na mesma jogada o jogador consegue eliminar uma sequência que vale 200 pontos, seguida de outra sequência que vale 250 pontos e finalmente uma terceira que vale 200: o jogador irá obter 10 pontos por uma jogada válida efetuada, acrescidos de 200 pontos da primeira eliminação, acrescidos de 250×2 pontos da segunda eliminação, acrescidos de 200×3 pontos da terceira eliminação em cadeia, perfazendo um total de 1310 pontos nesta jogada.

NOTA 2: Exemplos de eliminação e acomodamento simples

Exemplos A: um passo de eliminação

Antes

—
—
—
—
#
!
!
!
\$
#

Depois

—
—
—
—
#
—
—
—
\$
#

Antes

—
#
#
!
!
!
!
!
\$
#

Depois

—
#
#
—
—
—
—
—
\$
#

Exemplos B: um passo de acomodação

Antes

—
—
#
—
—
—
\$
#

Depois

—
—
—
—
#
\$
#

Antes

—
#
#
—
—
—
—
—

Depois

—
—
—
—
—
#
#

NOTA 3: Como são feitas eliminações e acomodações na vertical e na horizontal, a ordem pela qual são feitas interessa, assim como também interessa se se começa por fazer eliminações horizontais e a seguir verticais ou vice-versa.

Na classe `PlayArea` (fornecida por nós; já não precisa de fazer), as regras usadas para as eliminações e acomodações são as seguintes:

- primeiro na horizontal, de cima para baixo, em cada linha da esquerda para a direita;
- quando não é possível eliminar na horizontal, será na vertical, da esquerda para a direita, em cada coluna de cima para baixo.
- Após cada eliminação é feita a acomodação de símbolos suspensos.

Exemplo: as várias etapas de uma jogada

Passo 1

_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	!	_	_	\$	\$	_	_	_	_	_	_	_
_	_	_	!	_	_	!	#	_	_	_	_	_	_	_
_	_	_	\$	\$	_	\$	!	_	_	_	_	_	_	_
_	_	_	!	!	\$	%	#	_	_	_	_	_	_	_
_	_	_	#	#	\$	#	%	_	_	_	_	_	_	_

Passo 2
(entrou peça na coluna 6)

_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	!	_	\$	\$	\$	_	_	_	_	_	_	_
_	_	_	!	_	\$	!	#	_	_	_	_	_	_	_
_	_	_	\$	\$	\$	\$	!	_	_	_	_	_	_	_
_	_	_	!	!	\$	%	#	_	_	_	_	_	_	_
_	_	_	#	#	\$	#	%	_	_	_	_	_	_	_

Passo 3
(eliminação de sequencia horizontal)

_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	!	_	_	!	_	_	_	_	_	_	_	_
_	_	_	!	_	\$	!	#	_	_	_	_	_	_	_
_	_	_	\$	\$	\$	\$	!	_	_	_	_	_	_	_
_	_	_	!	!	\$	%	#	_	_	_	_	_	_	_
_	_	_	#	#	\$	#	%	_	_	_	_	_	_	_

Passo 4
(acomodação)

_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	!	_	_	!	_	_	_	_	_	_	_	_
_	_	_	!	_	\$	!	#	_	_	_	_	_	_	_
_	_	_	\$	\$	\$	\$	!	_	_	_	_	_	_	_
_	_	_	!	!	\$	%	#	_	_	_	_	_	_	_
_	_	_	#	#	\$	#	%	_	_	_	_	_	_	_

Passo 5
(eliminação de sequencia horizontal)

_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	!	_	_	!	_	_	_	_	_	_	_	_
_	_	_	!	_	\$	!	#	_	_	_	_	_	_	_
_	_	_	_	_	_	!	_	_	_	_	_	_	_	_
_	_	_	!	!	\$	%	#	_	_	_	_	_	_	_
_	_	_	#	#	\$	#	%	_	_	_	_	_	_	_

Passo 6
(acomodação)

_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	_	_	_	_	_	_	_	_	_	_	_	_
_	_	_	!	_	_	!	_	_	_	_	_	_	_	_
_	_	_	!	_	\$	!	!	_	_	_	_	_	_	_
_	_	_	!	!	\$	%	#	_	_	_	_	_	_	_
_	_	_	#	#	\$	#	%	_	_	_	_	_	_	_

							!					
							#					
				\$	!	!						
			!	\$	%	#						
		#	#	\$	#	%						

						!				
						#				
				\$	!	!				
			!	\$	%	#				
		#	#	\$	#	%				

[illegible][illegible]

Já sabem que para testarem as vossas classes devem usar a classe `PCOMain`.

Um *zip* contendo:

- 9

O que entregar?

Não há relatório a entregar porque o vosso *software* é a vossa documentação. Assim, têm que comentar condignamente as vossas classes: incluir no início de cada classe um cabeçalho Javadoc com `@author` (número do grupo e nome e número dos alunos que compõem o grupo); para cada método definido, incluir um cabeçalho incluindo a sua descrição, e, se for caso disso, `@param`, `@requires` e `@return`.

Para entregar: Um ficheiro *zip* com as classes e interfaces que compõem a vossa solução (somente com as classes que vos pedimos para fazerem).

O nome do ficheiro *zip* que contém o vosso trabalho deverá ter o formato `PCOxxx.zip` (onde `xxx` é o número do vosso grupo).

Como entregar o trabalho?

Através do Moodle de PCO. Às 23h55 do dia acordado para a entrega, 4 de Dezembro, os trabalhos entregues serão recolhidos.

Atenção que ao entregar o trabalho está a comprometer-se com o seguinte:

- O trabalho entregue é atribuível única e exclusivamente aos elementos que constituem o seu grupo;
- Qualquer indício de plágio será investigado e poderá levar ao não aproveitamento dos elementos do grupo e consequente processo disciplinar.